

Gojo — Project Documentation Report

An Airbnb-Style Booking Platform Server-Side Rendered Web Application using Node.js, Express, EJS, and PostgreSQL

1. Introduction

Gojo is a server-side rendered (SSR) booking platform inspired by Airbnb. It allows hosts to list properties for rent and guests to browse, search, book, and review those properties. The application was built to satisfy the core requirements of the Node.js + Express + EJS + PostgreSQL group assignment: a working Express server, route-based architecture, middleware usage, full CRUD operations, dynamic EJS templating with reusable partials, and a PostgreSQL database backing all data.

The platform supports two user roles — **hosts**, who create and manage property listings, and **guests**, who browse, book, and review stays — with role-based access control enforced through middleware.

2. Objectives

The project aims to demonstrate:

- A complete Express.js backend with organized routing and middleware
 - Server-side rendering using EJS, including partials and layout reuse (the assigned research component)
 - Full CRUD functionality backed by a PostgreSQL database
 - Secure authentication using hashed passwords and persistent sessions
 - A usable, responsive interface supporting the full booking lifecycle: browse, book, confirm, and review
 - Collaborative development practices using Git and GitHub, including individual contributions tracked through commits and pull requests
-

3. Technology Stack

Layer	Technology	Purpose
Runtime	Node.js (v18+)	JavaScript runtime for the server

Layer	Technology	Purpose
Framework	Express.js	Routing, middleware, request handling
Templating	EJS	Server-side HTML rendering with partials
Database	PostgreSQL (v14+)	Persistent storage for all application data
Authentication	bcryptjs	Password hashing
Sessions	express-session + connect-pg-simple	Session management, stored in PostgreSQL
Forms	method-override	Enables PUT/DELETE from HTML forms
Configuration	dotenv	Environment variable management
Frontend	CSS3, vanilla JavaScript, Google Fonts	Styling and minor client-side interactivity

No ORM is used — all database access is performed with parameterized raw SQL queries via the `pg` library, which keeps the data layer transparent and directly demonstrates SQL knowledge.

4. System Architecture

The application follows a standard SSR request flow:

1. The **browser** sends an HTTP request (e.g., visiting a property page, submitting a booking form).
2. **Express** receives the request and passes it through a chain of **middleware** — session handling, authentication checks (`requireAuth`, `requireHost`), and a `setLocals` middleware that loads the current user into every view.
3. The matching **route handler** (in `routes/`) queries **PostgreSQL** using parameterized SQL.
4. The route renders an **EJS template** (in `views/`), passing in the queried data.
5. EJS templates include shared **partials** (header, navbar, footer) for consistent layout across pages.
6. The fully rendered HTML page is sent back to the browser.

- **users** — id, name, email, hashed password, role ((host) or (guest))
- **properties** — id, host_id, title, description, price, location, bedrooms, bathrooms, max_guests, image_url, amenities
- **bookings** — id, guest_id, property_id, check_in, check_out, total_price, status ((pending) / (confirmed))
- **reviews** — id, user_id, property_id, booking_id, rating, comment
- **wishlists** — id, user_id, property_id (added as an additional feature — see Section 7)

Foreign keys link bookings and reviews to both users and properties, ensuring referential integrity.

5. Core Features

5.1 Authentication

Users register and log in through (routes/auth.js). Passwords are hashed with bcrypt before storage and never stored in plain text. On successful login, the user's ID and role are stored in (req.session), with sessions persisted in PostgreSQL via (connect-pg-simple) so users remain logged in across server restarts.

5.2 Property Listings (Full CRUD)

Hosts can create, view, edit, and delete property listings via (routes/properties.js):

- **Create** — (GET /properties/new) displays the listing form; (POST /properties) inserts a new row.
- **Read** — (GET /properties) lists and filters all properties; (GET /properties/:id) shows a single property with reviews and booking option.
- **Update** — (GET /properties/:id/edit) shows the edit form; (PUT /properties/:id) updates the row.
- **Delete** — (DELETE /properties/:id) removes the listing.

All write operations are protected by (requireAuth) and (requireHost), and hosts can only modify properties they own (enforced by checking (host_id) against the session user).

5.3 Browse & Search

The properties index page ((GET /properties)) supports filtering by location, minimum/maximum price, and guest capacity, dynamically building a parameterized SQL query based on which filters are supplied.

Guests reserve stays through `routes/bookings.js`. Bookings start in a `pending` state and hosts can confirm them from their dashboard. The dashboard separates "Pending Requests" (host view) from "My Bookings" (guest view), with notification badges for new activity.

5.5 Reviews & Ratings

After a confirmed stay, guests can leave a star rating and written comment via `routes/reviews.js`. Reviews are displayed on the property's detail page alongside the average rating context.

5.6 Dashboard

A role-aware dashboard (`views/dashboard.ejs`) shows hosts their listed properties, pending booking requests, and booking history, while guests see their bookings and recent confirmations.

5.7 EJS Partial and Layout Reuse

As required by the assignment's research component, the application uses EJS partials (`views/partials/header.ejs`, `navbar.ejs`, `footer.ejs`) included via `<%-include('partials/...') %>` across every page. This avoids duplicating layout markup and keeps navigation, branding, and notifications consistent application-wide.

6. Middleware Usage

Middleware	Location	Purpose
<code>express-session</code> + <code>connect-pg-simple</code>	<code>server.js</code>	Maintains login sessions, stored in PostgreSQL
<code>methodOverride('_method')</code>	<code>server.js</code>	Allows forms to send PUT/DELETE requests
<code>requireAuth</code>	<code>middleware/auth.js</code>	Blocks access to a route unless the user is logged in
<code>requireHost</code>	<code>middleware/auth.js</code>	Blocks access unless the user is logged in and has the <code>host</code> role
<code>setLocals</code>	<code>middleware/auth.js</code>	Runs on every request; loads the current user and notification counts

Middleware	Location	Purpose
		into <code>res.locals</code> so all views can access <code>currentUser</code>
Error-handling middleware	<code>server.js</code>	Catches errors from any route and renders a friendly error page with the appropriate status code

7. Additional Feature: Wishlist

Beyond the core requirements, a **Wishlist** feature was added, allowing guests to save properties of interest and revisit them later — a common pattern in real-world booking platforms (e.g., Airbnb's "Save" or Booking.com's "Save to a trip").

7.1 Motivation

The feature was chosen because it is self-contained (a new table, route file, and view, with minimal changes to existing files), demonstrates the full CRUD pattern using PostgreSQL, and adds genuine user value without risking conflicts with the rest of the codebase.

7.2 Database Change

A new `wishlists` table was added (`db/wishlist.sql`):

```
sql
CREATE TABLE IF NOT EXISTS wishlists (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  property_id INTEGER NOT NULL REFERENCES properties(id) ON DELETE CASCADE,
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE (user_id, property_id)
);

CREATE INDEX IF NOT EXISTS idx_wishlists_user ON wishlists(user_id);
```

The `UNIQUE(user_id, property_id)` constraint ensures a guest cannot save the same property twice at the database level. `ON DELETE CASCADE` ensures that if a user or property is deleted, related wishlist entries are automatically removed, preventing orphaned records.

7.3 New Routes — `routes/wishlists.js`

Method	Route	Description
GET	<code>/wishlists</code>	Fetches all properties saved by the logged-in user (joins <code>wishlists</code> with <code>properties</code>) and renders the "My Wishlist" page
POST	<code>/wishlists/:propertyId</code>	Inserts a new wishlist row for the current user and property; uses <code>ON CONFLICT (user_id, property_id) DO NOTHING</code> to safely ignore duplicate saves
DELETE	<code>/wishlists/:propertyId</code>	Removes the wishlist entry for the current user and property

All three routes are protected by the existing `requireAuth` middleware, following the same pattern as bookings and reviews.

7.4 View and UI Changes

- **New page** — `views/wishlists/index.ejs` displays saved properties using the existing `.property-card` grid layout, each with a "Remove from Wishlist" button.
- **Property detail page** — `views/properties/show.ejs` gained a "Save to Wishlist" / "Remove from Wishlist" button in the booking sidebar, which toggles based on whether the property is already saved.
- `routes/properties.js` — the `(GET /properties/:id)` route was extended with a query against the `wishlists` table to determine whether the current user has already saved the property, passing an `isWishlisted` flag to the view.
- **Navbar** — `views/partials/navbar.ejs` gained a "Wishlist" link for logged-in users, alongside "Bookings".

7.5 End-to-End Flow

1. A logged-in guest clicks "**Save to Wishlist**" on a property's detail page.
2. The browser sends `(POST /wishlists/:propertyId)`.
3. The server inserts a row into `wishlists` linking `user_id` and `property_id` (or does nothing if it already exists).
4. The page reloads and the button now reads "**Remove from Wishlist**".
5. Clicking "**Wishlist**" in the navbar sends `(GET /wishlists)`, which joins `wishlists` with `properties` and displays all saved listings in a grid.

- Clicking **"Remove from Wishlist"** sends `DELETE /wishlists/:propertyId`, deleting the row and removing the property from the list.
-

8. Installation and Setup

Prerequisites

- Node.js v18 or higher
- PostgreSQL v14 or higher
- npm (bundled with Node.js)

Steps

1. Clone the repository

```
bash

git clone https://github.com/tenuleta/Gojo.git
cd Gojo
```

2. Install dependencies

```
bash

npm install
```

3. Set up the database Create the `gojo` database, then run the schema and seed files:

```
bash

psql -U postgres -d gojo -f db/schema.sql
psql -U postgres -d gojo -f db/seed.sql
psql -U postgres -d gojo -f db/wishlist.sql
```

4. Configure environment variables Create a `.env` file in the project root:

```
env

DATABASE_URL=postgresql://postgres:<your_password>@localhost:5432/gojo
```

5. Start the application

```
bash
```

```
npm start
```

6. Open the browser Visit `http://localhost:3000`

Demo Accounts

Name	Email	Password	Role
Alice Kamau	alice@example.com	password123	Host
Bob Ochieng	bob@example.com	password123	Host
Carol Wanjiku	carol@example.com	password123	Guest
David Mwangi	david@example.com	password123	Guest
Eve Akinyi	eve@example.com	password123	Host

9. Project Structure

```
gojo/
├── server.js                # Express app entry point
├── package.json
├── .env                    # Environment variables (not committed)
├── config/
│   └── db.js               # PostgreSQL connection pool
├── middleware/
│   └── auth.js             # requireAuth, requireHost, setLocals
├── routes/
│   ├── index.js           # Home page + Dashboard
│   ├── auth.js            # Register, login, logout
│   ├── properties.js      # Property CRUD
│   ├── bookings.js        # Booking management
│   ├── reviews.js         # Review creation
│   └── wishlists.js        # Wishlist add/remove/view (new)
├── views/
│   ├── partials/          # EJS partials (header, footer, navbar)
│   ├── index.ejs          # Home page
│   └── dashboard.ejs      # User dashboard
```



```
|   |─ bookings.ejs           # Booking list
|   |─ error.ejs             # Error page
|   |─ auth/                 # Login and register pages
|   |─ properties/           # Property list, detail, create, edit
|   |─ wishlists/            # My Wishlist page (new)
|─ public/
|   |─ css/style.css         # Stylesheet
|   |─ js/main.js            # Client-side JavaScript
|─ db/
|   |─ schema.sql            # Database schema
|   |─ seed.sql              # Sample data
|   |─ wishlist.sql          # Wishlist table (new)
```

10. Version Control and Collaboration

The project was developed collaboratively on GitHub, with all group members contributing through commits to the shared repository.

The Wishlist feature was developed on a separate branch (`feature/wishlist`) to avoid disrupting the main codebase:

```
bash

git checkout -b feature/wishlist
git add .
git commit -m "Add wishlist feature: save/remove properties and view saved list"
git push -u origin feature/wishlist
```

A Pull Request was then opened against (`main`), allowing the change to be reviewed before merging — keeping the contribution clearly attributed and the commit history meaningful, as required by the assignment.

11. Conclusion

Gojo satisfies all core technical requirements of the assignment: an Express server with organized routing and middleware, full CRUD operations across multiple entities, dynamic EJS rendering with reusable partials, and a PostgreSQL-backed data layer accessed through parameterized queries. The platform supports a complete booking lifecycle — from browsing and searching properties, through booking and confirmation, to leaving reviews — with role-based access for hosts and guests.

The Wishlist feature extends the platform with an additional, self-contained piece of functionality that mirrors a real-world booking-site capability, implemented end-to-end (database table, routes, views, and UI integration) and delivered through a standard Git branch-and-pull-request workflow.